

ContractBPF-Ledger: Bounded Resource-Effect Accounting for BPF-Programmable Scheduling and Paging

Paper ID XXX
Anonymous Institution

Abstract

Networked services increasingly rely on programmable kernel policies to meet workload-specific latency, throughput, and resource-efficiency goals. eBPF and related mechanisms now make it possible to specialize core policy paths such as scheduling and memory management. Existing verifier, isolation, and language-based mechanisms protect important local safety properties, including memory safety, bounded control flow, helper-call validity, and pointer safety. They do not directly reason about the dynamic resource effects of multiple independently loaded policies. A BPF scheduler and a BPF paging policy may both pass verification and improve their own objectives in isolation, yet their composition can create refault storms, queue-delay amplification, and tail-latency collapse for a multi-tenant networked service.

This paper presents ContractBPF-Ledger, a resource-effect accounting system for BPF-programmable scheduling and paging policies. ContractBPF-Ledger grants policies effect tokens, records their resource effects in per-scope ledgers, and enforces budgets at effect boundaries rather than at every BPF instruction. When a policy exceeds its budget or triggers a cross-subsystem feedback loop, ContractBPF-Ledger performs bounded degradation: it throttles or revokes the smallest harmful effect, such as page demotion, while preserving unrelated safe policy functionality.

We implement ContractBPF-Ledger on Linux using `sched_ext` and a conservative PageFlex-style paging decision hook. The current artifact validates the mechanism in QEMU: a controlled G1–G9 experiment matrix reproduces the scheduler-paging feedback loop and shows recovery by revoking only `demote_page`, while a memcached G1–G9 companion matrix demonstrates the same artifact path on a real networked service. These QEMU results are correctness and reproducibility evidence, not production performance claims.

1 Introduction

Cloud and multi-tenant networked services are increasingly sensitive to kernel resource policy. A small shift in scheduling, memory reclaim, page demotion, or NUMA placement can be the difference between stable tail latency and service-level objective violations. At the same time, operators want policies that are specialized to their own services, hardware, and workload phases. This has made kernel programmability

attractive: eBPF programs can encode service-specific logic and run inside privileged kernel paths after verifier checks.

This trend is moving beyond packet filters and tracing. `sched_ext` allows scheduling behavior to be defined by BPF programs. PageFlex-style mechanisms show that paging decisions can also be delegated or customized through BPF-assisted policy paths. These mechanisms let operators specialize scheduling and memory policy for networked services, but they also introduce a new failure mode: independently safe policies may create harmful resource effects when composed.

Consider a latency-sensitive key-value service under memory pressure. A BPF scheduler boosts the service when queue delay increases. A BPF paging policy, using stale phase information, demotes pages from the same service. Each policy is reasonable alone: the scheduler reduces queueing, and the paging policy reduces memory footprint. Together, however, they can form a feedback loop. The scheduler gives the service more execution opportunities; the service touches pages that the paging policy demoted; refaults increase; queue delay and tail latency rise; the scheduler boosts again; the paging policy continues demoting. The result is a verifier-accepted but harmful composition.

The key observation is that verifier acceptance is not resource-effect safety. The verifier can reject unsafe memory accesses or unbounded control flow, but it does not prove that a policy creates only a bounded amount of queueing, demotion, refault, or cross-subsystem debt for a service scope. Nor does it specify how a system should recover when a policy remains memory-safe but becomes effect-unsafe.

We propose *resource-effect safety* as a missing safety dimension for programmable kernel policy. Resource-effect safety asks three questions. First, which resource effects may a policy create? Second, how much resource debt may each effect create within a service scope? Third, when a budget is violated, can the system recover by revoking only the harmful effect rather than disabling all BPF functionality?

ContractBPF-Ledger answers these questions using three mechanisms. First, *effect tokens* authorize a BPF policy to create specific effects, such as `boost_task` or `demote_page`, within a scope and budget. Second, *per-scope resource ledgers* charge scheduler and paging effects to shared service scopes, connecting cgroup-level scheduling events and memcg-level memory events. Third, *bounded degradation* throttles or revokes the smallest harmful effect that explains a violation.

This paper targets NSDI because the motivating problem is not merely kernel extensibility. It is the stability of networked

services whose resource-management policies are increasingly programmable. The current artifact starts with controlled QEMU validation and a memcached G1–G9 companion matrix; a submission-ready evaluation must still extend this to non-QEMU latency-sensitive services, multi-tenant pressure, and service-level recovery.

Contributions. This paper makes four contributions:

- We identify resource-effect conflicts among verifier-accepted BPF policies as a safety problem for programmable resource management in networked services.
- We define effect tokens and per-scope ledgers for accounting scheduling and paging effects across shared service scopes.
- We design effect-boundary enforcement and bounded degradation, allowing the system to revoke harmful effects while preserving unrelated safe policy functionality.
- We implement ContractBPF-Ledger on Linux using `sched_ext` and a conservative PageFlex-style paging decision hook, and validate it with QEMU scheduler-paging conflicts plus a memcached service matrix.

2 Background and Motivation

2.1 BPF safety today

The eBPF verifier is a gatekeeper for kernel-resident BPF programs. It reasons about program control flow, bounded execution, register and stack state, pointer types, helper arguments, and memory accesses. This prevents many classes of unsafe programs from entering the kernel. However, verifier acceptance does not imply that the policy encoded by a program is benign under every runtime condition. A scheduler policy can legally reorder tasks in a way that increases starvation risk. A paging policy can legally request demotion decisions that later increase refaults. These are policy-effect failures rather than verifier failures.

2.2 Programmable scheduling

`sched_ext` exposes a BPF-defined scheduler interface. A BPF scheduler can decide how tasks are enqueued, dispatched, and selected for CPUs. This makes it possible to specialize scheduling policy for service-level objectives, but it also allows policies to create resource effects such as boosting, pinning, dispatch failures, and queue-delay shifts.

2.3 Programmable paging

PageFlex-style systems show that paging policy decisions can be delegated while exposing kernel memory state at low overhead. This creates a complementary source of resource effects: demotion, reclaim hints, region classification, refault amplification, and fault-latency changes.

2.4 The missing cross-subsystem layer

The scheduler and memory manager are coupled through workload execution. A scheduling decision changes which pages are touched. A paging decision changes how quickly tasks can make progress. If both sides become programmable, local safety is insufficient. We need a way to account for cross-subsystem effects at the service scope where the application experiences them.

3 Problem Statement

3.1 Threat model

We assume BPF programs pass the Linux verifier. We do not address verifier soundness bugs, JIT bugs, arbitrary kernel memory corruption, or hardware side channels. We focus on policies that are locally legal but may be malicious, mis-configured, stale, or generated by independent controllers with conflicting objectives. Policies may belong to different teams or tenants.

3.2 Resource-effect conflict

A resource-effect conflict occurs when two or more legal policy effects jointly violate a runtime resource invariant. In this paper we focus on scheduler-paging conflicts:

A scheduler effect increases execution opportunities for a service scope while a paging effect increases refault or fault-latency debt for the same scope, causing queue-delay and tail-latency amplification.

3.3 Design goals

1. **Post-verifier enforcement.** Do not replace the verifier.
2. **Effect-level attribution.** Attribute debt to effects, not just programs.
3. **Cross-subsystem scopes.** Account scheduler and paging effects in the same service scope.
4. **Low overhead.** Enforce at effect boundaries, not every instruction.
5. **Bounded degradation.** Prefer throttling or revoking harmful effects over killing the whole policy.

4 Design

4.1 Overview

ContractBPF-Ledger consists of a user-space contract manager and a kernel resource-effect layer. The contract manager parses manifests, installs effect tokens, resolves scopes, and exports audit events. The kernel layer stores token tables,

Table 1: Examples of resource effects.

Subsystem	Effect	Resource debt
sched_ext	boost_task	queue delay, boost rate
sched_ext	dispatch_task	dispatch failures
sched_ext	pin_cpu	CPU imbalance, starvation
Paging	demote_page	refaults, fault latency
Paging	reclaim_hint	reclaim pressure
Paging	classify_region	stale phase decisions

updates per-scope ledgers, checks effect gates, and triggers degradation.

4.2 Effect tokens

An effect token authorizes a resource effect within a scope and budget:

$Token = \langle subsystem, effect, scope, budget, fallback \rangle$.

For example, a paging policy may receive a token authorizing `demote_page` for `memcg` A with a refault budget and a `revoke_demote_page` fallback.

This differs from helper permissions. A helper allowlist asks whether a program may call an API. An effect token asks whether a policy may create a bounded resource effect for a given service scope.

4.3 Per-scope resource ledgers

A ledger aggregates resource debt for a scope:

```
struct ledger {
    scope_id scope;
    u64 sched_queue_delay_us;
    u64 sched_dispatch_failures;
    u64 sched_boost_events;
    u64 pages_demoted;
    u64 refault_events;
    u64 major_fault_events;
    u64 fault_latency_us;
    u32 degrade_state[EFFECT_MAX];
};
```

The scope can be a cgroup, memory cgroup, CPU set, NUMA node, or memory region. The paper focuses on service scopes that connect cgroup and memcg identities.

4.4 Effect-boundary enforcement

ContractBPF-Ledger checks only when a BPF policy attempts to create a resource effect. For `sched_ext`, boundaries include `select_cpu`, `enqueue`, `dispatch`, `boost`, and CPU pinning. For paging, boundaries include demotion decisions, reclaim hints, and region classification.

```
int gate(prog, effect, scope, cost) {
    token = lookup_token(prog, effect, scope);
    if (!token) return DENY;
```

Table 2: Bounded degradation levels.

Level	Action	Example
0	normal execution	allow boost and demotion
1	throttle effect	reduce demotion rate
2	revoke effect	revoke <code>demote_page</code>
3	subsystem fallback	default kernel reclaim
4	disable policy	disable <code>sched_ext</code> policy

```
ledger = lookup_ledger(scope);
if (would_exceed(ledger, token, cost)) {
    trigger_degrade(prog, effect, scope, ledger);
    return DEGRADED;
}

charge(ledger, effect, cost);
return ALLOW;
```

4.5 Cross-subsystem rule

The minimal rule used in the prototype is:

If a service scope has high refault ratio, high queue delay, and high demotion rate in the same epoch window, revoke the `demote_page` effect for that scope while preserving scheduler dispatch.

This rule is intentionally simple. The paper’s claim is not that this is the only possible invariant, but that a small set of resource-effect invariants can prevent damaging feedback loops.

4.6 Bounded degradation

The key design choice is minimality. If refault debt is caused by page demotion, ContractBPF-Ledger revokes demotion before disabling the scheduler. This preserves useful BPF functionality and reduces disruption.

5 Implementation

We implement ContractBPF-Ledger in Linux using three components: a contract manager, a `sched_ext` front-end, and a paging front-end.

The artifact pins Linux 6.12.30 and keeps the kernel changes as a patch series under `kernel/patches/`. The current series contains 2,294 patch lines across the core token and ledger layer, `sched_ext` gate, conservative MM decision hook, cross-subsystem rule, and `kselftest` wiring. The repository also contains QEMU rootfs builders, run scripts, BPF policy stubs, Rust user-space tools, workload drivers, and experiment parsers. Generated Linux trees, initramfs images, raw logs, and figures are kept out of version control.

5.1 Contract manager

The user-space manager parses YAML manifests, validates effect names and fallback availability, resolves scopes, and installs token maps. It also streams violation events for evaluation.

5.2 sched_ext integration

The current sched_ext front-end gates the dispatch/boost path used by the prototype scheduler. It records queue delay, boost rate, and dispatch activity using per-scope counters. The broader design also targets select_cpu, enqueue, and CPU-pinning boundaries, but those are not yet all implemented in the artifact.

5.3 Paging integration

The paging front-end follows a conservative decision-only design. BPF programs receive summarized read-only state and return keep, demote, reclaim_hint, or no_op. The kernel validates page state and effect budgets before execution.

5.4 Overhead controls

ContractBPF-Ledger uses per-CPU counters, epoch aggregation, static token lookup caching, and optional sampling for high-rate paging events. These choices are necessary to keep enforcement overhead low.

6 Prototype Evaluation

This section reports the evidence currently produced by the artifact. All experiments below run in QEMU on the pinned Linux 6.12.30 kernel. We use QEMU to validate correctness and reproducibility of the kernel paths; the absolute latency and throughput numbers are not production performance results.

6.1 Workloads

The main matrix uses a controlled phase-changing service that touches memory in phases and emits per-phase latency samples. This workload is intentionally small so that every baseline can run in one QEMU boot and produce comparable raw serial evidence. The artifact also includes a real memcached path: make memcached-experiments boots QEMU, starts two memcached instances in the guest, and runs an ASCII protocol load client across the same G1–G9 baseline set. This matrix uses controlled kernel effect injection for the scheduler-paging conflict; it is a real-service artifact validation, not a production performance experiment.

Table 3: Evaluation baselines.

Group	Configuration
G1	default Linux scheduler and paging
G2	sched_ext policy only
G3	BPF paging policy only
G4	sched_ext + BPF paging without ContractBPF-Ledger
G5	cgroup/memcg quota-style controls
G6	static checker only
G7	per-subsystem ledger only
G8	kill-whole-policy fallback
G9	full ContractBPF-Ledger

Table 4: Controlled QEMU matrix summary. Latencies are guest synthetic-service samples.

Group	P99 us	Queue delay us	Demoted	Demote state
G1	102716	0	0	0
G2	98907	15703000	0	0
G3	103104	0	3	4
G4	95487	74000	8	0
G5	100191	0	0	0
G6	81575	0	0	0
G7	97912	11076000	3	4
G8	97349	170000	8	0
G9	96559	73000	8	2

6.2 Baselines

6.3 Metrics

We measure P50/P99/P999 latency, throughput, queue delay, boost events, pages demoted per epoch, refault ratio, major fault rate, fault latency, fallback activation latency, recovery time, and steady-state overhead.

6.4 Controlled QEMU matrix

The matrix produces raw logs under experiments/results/raw/, processed CSV tables under experiments/results/processed/, and five SVG figures: feedback timeline, tail latency, recovery, ablation, and overhead. In the controlled conflict, G4 records an unguarded combined scheduler-paging case with page demotion and refaults but no demote-page degradation. G9 records the full ContractBPF-Ledger path: the same demotion/refault condition reaches demote degrade state 2, which corresponds to revoking demote_page, while preserving scheduler dispatch. The recovery table marks G9 as recovered in one epoch and the whole-policy fallback as two epochs.

6.5 Real-service companion matrix

The memcached matrix proves that the artifact can boot a guest, start real networked services, drive load, and collect latency samples across every baseline group. The raw log is the May 22 memcached-matrix QEMU log, and the processed table is memcached_matrix.summary.csv. The controlled conflict counters follow the same pattern as the synthetic matrix: G4 records demotion and refaults with no

Table 5: QEMU memcached companion matrix.

Group	P99 us	Tenant-B P99	Queue us	Demoted	Demote state
G1	1608	7482	0	0	0
G2	969	2508	0	0	0
G3	1221	1138	0	3	4
G4	1463	1787	12000	8	0
G5	1074	998	0	0	0
G6	867	2936	0	0	0
G7	809	3153	17109000	3	4
G8	839	1630	12000	8	0
G9	733	2732	77000	8	2

demote-page revocation, while G9 reaches demote degrade state 2 and records one-epoch recovery.

6.6 Current findings

1. The kernel patch series builds, QEMU boots, sched_ext loads and unloads, contractd starts, and kselftests exercise token, ledger, degrade, MM hook, and cross-subsystem paths.
2. The controlled unguarded scheduler-paging scenario records the feedback-loop counters needed for the thesis: queue delay, demotion, refaults, and no demote-page revocation.
3. The full ContractBPF-Ledger path detects the cross-subsystem condition and revokes only demote_page while keeping scheduler dispatch intact.
4. A real memcached service can run in QEMU across the full G1–G9 matrix and produce latency samples, but non-QEMU performance validation remains future work.

7 Discussion

7.1 Why not cgroups?

cgroups bound coarse resource usage but do not attribute debt to specific BPF effects. A cgroup CPU quota can throttle service execution, but it cannot revoke only a demote_page effect while preserving safe scheduler dispatch.

7.2 Why not static checking?

Static checks can reject obvious manifest conflicts, but the motivating failure depends on runtime phase, memory pressure, refaults, and queue delay. These are dynamic effects.

7.3 Why NSDI rather than a pure OS venue?

The mechanisms are kernel mechanisms, but the failure target is networked services. Programmable resource policies are becoming part of cloud service management, so a strong NSDI submission must emphasize multi-tenant service stability, tail latency, workload phases, and operational usefulness. The current artifact provides the kernel and QEMU evidence needed for that argument, but it still needs a broader real-service evaluation.

7.4 Limitations

ContractBPF-Ledger currently targets sched_ext and paging. It does not handle all BPF program types, does not replace the verifier, and does not prove global optimality of resource policies. Threshold selection and scope mapping require careful configuration.

The current evaluation is also limited. QEMU validates kernel control flow, reproducibility, and failure recovery, but it is not evidence of production performance. The paging path is a conservative kernel-validated prototype, not the final PageFlex-style BPF loading path. The memcached matrix is a real networked-service validation in QEMU with controlled effect injection; it is not yet a non-QEMU performance study under realistic multi-tenant memory pressure. The paper must not claim submission-ready performance results until those experiments are added.

8 Related Work

eBPF verifier. The verifier provides local execution safety for BPF programs [1]. ContractBPF-Ledger is complementary: it handles runtime resource effects after verifier acceptance.

sched_ext. sched_ext enables BPF-defined scheduling policies [2]. ContractBPF-Ledger adds bounded effect accounting for scheduling decisions.

BPF-assisted paging. PageFlex demonstrates flexible paging policy delegation using eBPF-style mechanisms [5]. ContractBPF-Ledger studies what happens when such paging policies interact with programmable scheduling.

Static eBPF policy enforcement. KRAKENGUARD enforces policy-driven constraints on BPF bytecode at load time and detects cross-program interference [4]. ContractBPF-Ledger instead targets runtime state-dependent resource-effect debt.

Safe kernel extensions. Rex addresses language and verifier usability for safe kernel extensions [3]. ContractBPF-Ledger addresses resource-effect conflicts even when code is safe.

9 Conclusion

BPF programmability is moving into core kernel policy paths that affect networked services. Local verifier acceptance is necessary but not sufficient when independently loaded policies create interacting resource effects. ContractBPF-Ledger introduces effect tokens, per-scope ledgers, and bounded degradation to account for and recover from scheduler-paging resource-effect conflicts. The central lesson is that programmable kernel policy needs runtime effect safety, not just program safety.

References

- [1] ebpf verifier. Linux Kernel Documentation, 2026. <https://docs.kernel.org/bpf/verifier.html>.
- [2] Extensible scheduler class. Linux Kernel Documentation, 2026. <https://docs.kernel.org/scheduler/sched-ext.html>.
- [3] Jinghao Jia et al. Rex: Closing the language-verifier gap with safe and usable kernel extensions. USENIX ATC 2025, 2025. IBM Research publication page.
- [4] Jay Patel et al. Krakenguard: Towards fine-grained ebpf isolation. In *23rd USENIX Symposium on Networked Systems Design and Implementation*, 2026. <https://www.usenix.org/conference/nsdi26/presentation/patel>.
- [5] Anil Yelam, Kan Wu, Zhiyuan Guo, Suli Yang, Rajath Shashidhara, Wei Xu, Stanko Novaković, Alex C. Snoeren, and Kimberly Keeton. Pageflex: Flexible and efficient user-space delegation of linux paging policies with ebpf. In *2025 USENIX Annual Technical Conference*, 2025. <https://www.usenix.org/conference/atc25/presentation/yelam>.